

CURSO TÉCNICO SUPERIOR PROFISSIONAL DE PROGRAMAÇÃO DE SISTEMAS DE INFORMAÇÃO
DESENVOLVIMENTO DE APLICAÇÕES

PADRÕES DE ARQUITETURA DE SOFTWARE

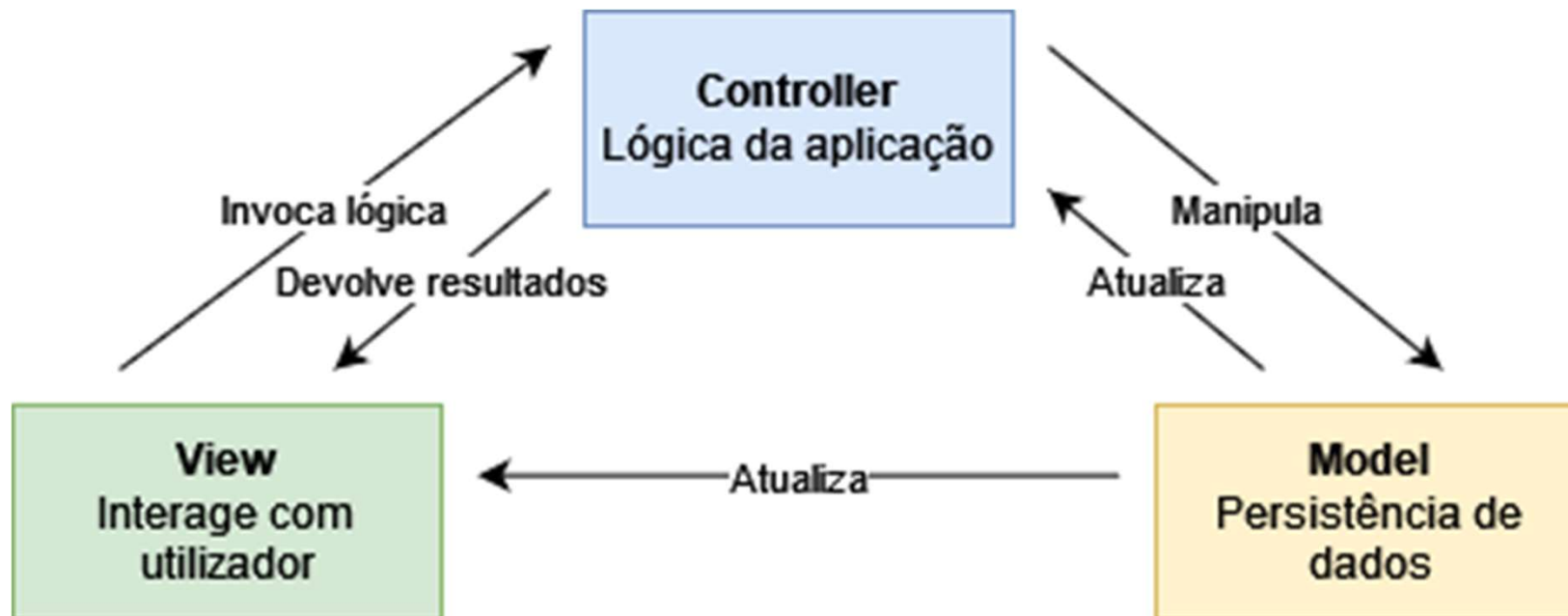
ARQUITETURA DE SOFTWARE

- Definem soluções recorrentes para problemas arquiteturais comuns
- Usados como referência durante o desenvolvimento de software
- Principais benefícios
 - Permite a reutilização de soluções, em vez de “reinventar a roda” constantemente
 - Por ex., utilização da mesma biblioteca em diferentes aplicações
 - Melhor qualidade de software
 - Desenvolvimento mais rápido
 - Manutenção mais fácil

PADRÕES DE ARQUITETURA COMUNS

- Model-View-Controller (MVC)
- Model-View-Presenter (MVP)
- Model-View-ViewModel (MVVM)
- Event-driven Architecture
- Service-Oriented Architecture (SOA)
- Microservices Architecture

MODEL-VIEW-CONTROLLER

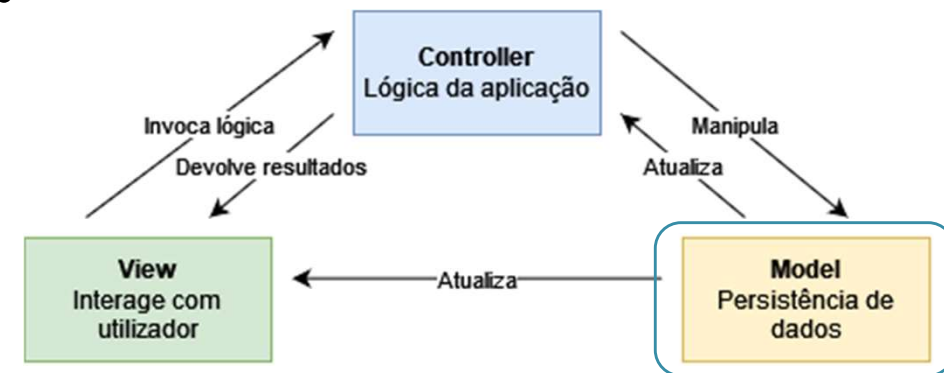


PRINCÍPIOS

- Separation of Concerns (“Separação de Preocupações”)
 - Define que o código relativo a uma responsabilidade do software deve ser segregado das restantes responsabilidades
- Don’t Repeat Yourself – DRY (Não te repitas)
 - Se um bloco de código é repetido em vários locais, este deve ser abstraído para ser implementado uma única vez e reutilizável onde necessário
- Separação de dados e UI
 - Os componentes de acesso a dados e lógica devem ser sempre separados da interface
- Single Responsibility (Responsabilidade única)
 - Cada componente deve ter apenas uma única tarefa ou funcionalidade

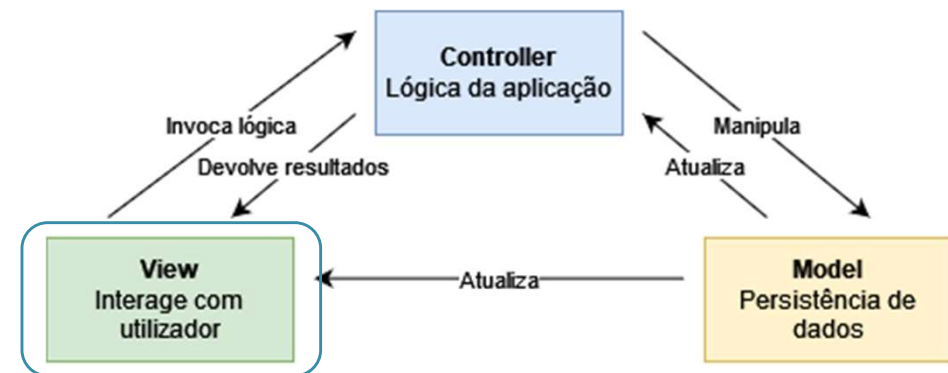
MVC – COMPONENTE MODEL

- Responsável pelo acesso e manipulação de dados
 - Define as regras de acesso e persistência de dados
 - Garante a lógica de negócio ao definir regras de validação e outros cálculos
 - Notifica os outros componentes de alterações
-
- Apenas pode ser acedido pelos outros componentes
 - **Não deve ter conhecimento nem da View nem do Controller**



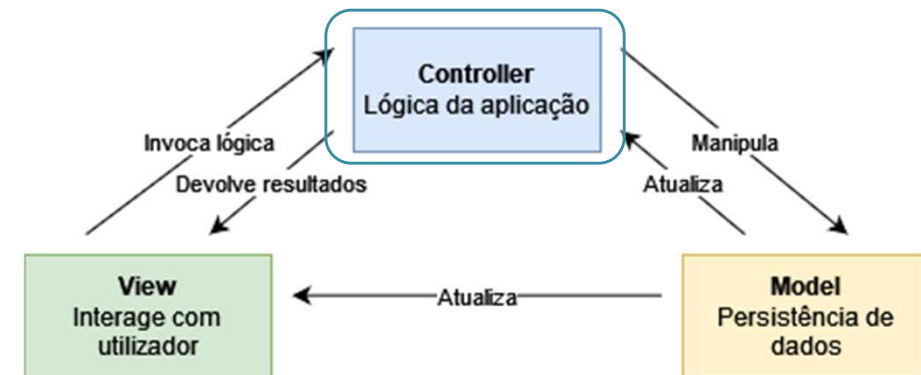
MVC – COMPONENTE VIEW

- Responsável pela apresentação ao utilizador
 - Deve conter apenas lógica de visualização
 - Recebe os inputs do utilizador e comunica-os ao Controller se necessário
-
- Pode requerer dados diretamente ao Model
 - **Nunca** deve manipular os dados do Model diretamente



MVC – COMPONENTE CONTROLLER

- Responsável por receber inputs da View e determinar a sua resposta
 - Manipula os dados do Model se necessário
 - Coordena as interações entre View e Model
 - Notifica a View de alterações no Model
-
- Garante a consistência entre a View e o Model:
 - ao manipular os dados do Model...
 - para responder aos pedidos da View



ARQUITETURAS DE SOFTWARE

- **Vantagens:**

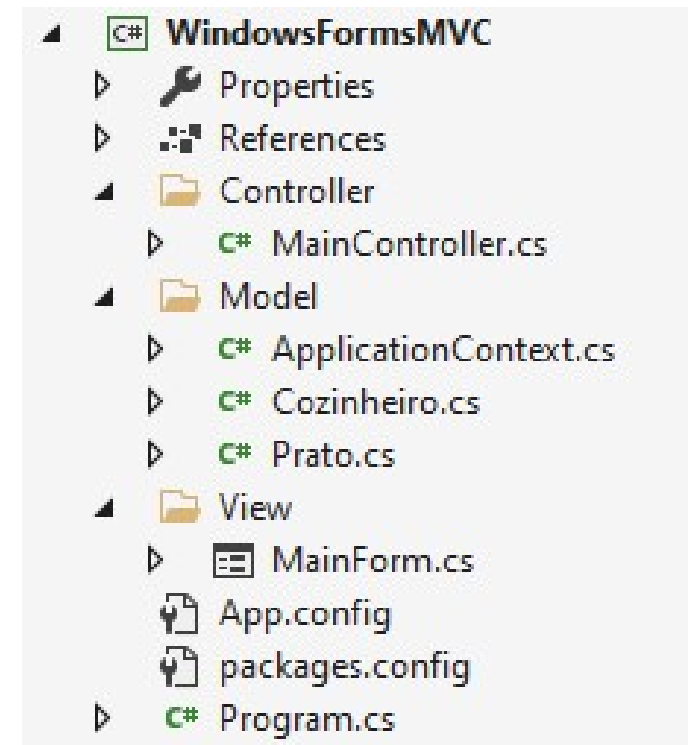
- Permite a fácil compreensão da estrutura do código
- Torna a expansão de funcionalidades mais simples
- Facilita a reutilização de código
- Aumenta a robustez e qualidade da aplicação
- Agiliza a resolução de problemas ou bugs que possam ocorrer

- **Desvantagens:**

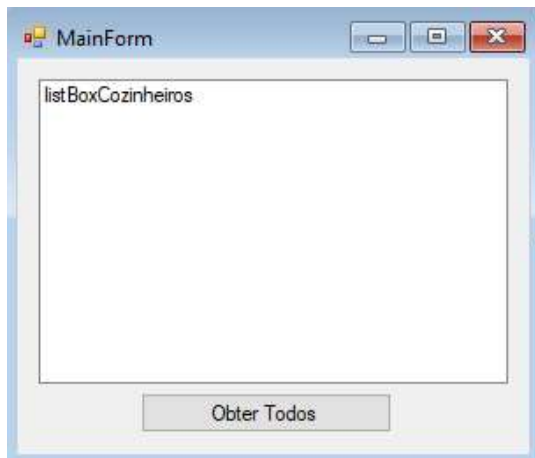
- Aumenta a complexidade de desenvolvimento inicial
- Nem todos os padrões funcionam em todas as situações
- Requer mais experiência da equipa de desenvolvimento

IMPLEMENTAÇÃO - ESTRUTURA

- Sugestão de estrutura para aplicação em .NET WinForms
- Controller irá conter a lógica da aplicação
 - Geralmente um Controller por cada Form
- Model terá as classes (entidades) e objetos de configuração do Entity Framework
- A diretoria View terá todos os formulários do programa



IMPLEMENTAÇÃO – VIEW



```
private void buttonGetCozinheiros_Click(...) {  
    var cozinheiros = MainController.GetCozinheiros();  
    listBoxCozinheiros.DataSource = cozinheiros;  
}
```

- Deve aceder ao Controller para obter os dados a mostrar ao utilizador
- Só permite lógica de apresentação

IMPLEMENTAÇÃO - CONTROLLER

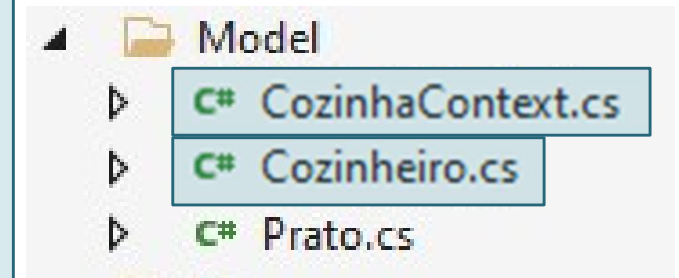
```
class MainController
{
    public static List<Cozinheiro> GetCozinheiros()
    {
        using (var db = new CozinhaContext())
        {
            return db.Cozinheiros.ToList();
        }
    }
    ...
}
```

- Contém a lógica de resposta aos inputs da View
- Obtém os dados do Model e converte-os para algo que seja útil à View

IMPLEMENTAÇÃO - MODEL

```
CozinhaContext.cs    public class CozinhaContext : DbContext
                      {
                        public DbSet<Cozinheiro> Cozinheiros { get; set; }
                        public DbSet<Prato> Pratos { get; set; }
                      }
```

```
Cozinheiro.cs        public class Cozinheiro
                      {
                        int Id { get; set; }
                        string Nome { get; set; }
                        List<Prato> Pratos { get; set; }
                      }
```



FONTES

- **“Design Patterns: Elements of Reusable Object-Oriented Software”** - Erich Gamma, Richard Helm, Ralph Johnson e John Vlissides
- **“Agile Principles, Patterns, and Practices in C#”** – Robert C. Martin e Micah Martin